

Variant Query Service — Hackathon API Reference

What is this service?

The **Variant Query Service (VQS)** is a query engine over genomic variant datasets produced by Sophia Genetics' NGS pipelines. Under the hood it is a multi-tenant wrapper around **DuckDB** that exposes a uniform JSON API for browsing, querying and filtering tabular data.

It can:

- **Sort, filter, paginate** large variant tables interactively
- **Join live data** (e.g. user annotations, phenotypic ranking) on top of the base dataset via composable *transformers*
- **Cache and materialize** datasets for faster repeated access
- **Export** filtered results as TSV / CSV
- Operate on **any tabular dataset**, not just variants — it is essentially "DuckDB as a service"

A query targets a **Dataset** identified by a **key** (resolved by the Genomic Data service in production), optionally chains one or more **transformers** that modify or enrich the data, and then runs a **query body** describing columns / filters / sort / pagination.

Useful resources for the hackathon

We provide over 30 different resources through VQS to the front-end, allowing also to join different internal and external data to the pipeline results.

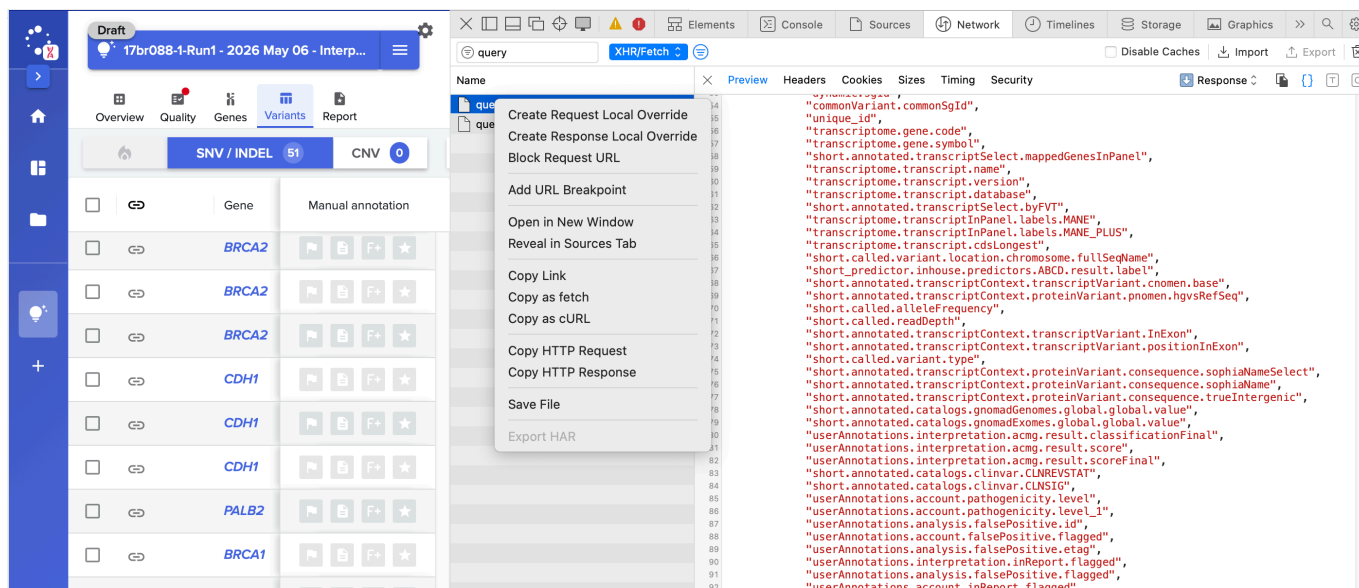
For the scope of the hackathon, the following are the most important queries:

- **Gene table:** Summarizes the variants that were found in the analysis, their classification and disease relevance
- **Variants table:** Shows the details on each of the variants (we provide over 500 points of information per variant in the data!)

The actual queries that are sent are depending on the analysis and interpretation. The best way to get started is to find a query of interest sent from the front-end, and modifying that.

Steps

1. Go to the variants or gene view table
2. Open the browser developer tools, check in the network tab
3. Clear any entries that are there, and scroll until you see a new query that was sent
4. Click to see the details and 'Copy as cURL'



Example query:

```
curl 'https://platform-vandv1.sophiagenetics.com/api/variant/query?
key=eyJmaWNJZCI6MjA2NTA3NTM4LCJrbm93bkZpbGUiOiJMQVlFUjJfU0hPUlRfU1VCU0VUX0R
BVEFTRVRfUEXVU19HRU5FX0F0Tk9UQVRJT05TIiwiaW5hbHlzaXNjZCI6MjAwMDQyMzYwLCJpbm
RlcnByZXRhZGlvbII6ImIudGVycHJldGF0aW9uLTEwNDIiLCJ0cmFuc2NyaXB0U3RyYXRlZ3lWZ
XJzaW9uIjoimTc3ODA2OTAYNjAwMCJ9&engine.paginate=true&plan.parameters.uaVari
antId=COMMON_SGID&plan.parameters.experimentTypeId=126000&plan.parameters.p
miId=200040484&plan.parameters.analysisId=200042360&plan.parameters.interpr
etationId=interpretation-
1042&plan.transformers=FLATTEN%2CUSER_ANNOTATIONS%2CPhenotypic_Ranking%2CUS
ER_ANNOTATIONS_REFRESH%2CPhenotypic_Ranking_REFRESH%2CShort_Subset&engine.m
axPaginationRows=500&plan.parameters.uaAcmgOverrideVersion=0&plan.parameter
s.uaAcmgCnvOverrideVersion=0&plan.parameters.uaVariantInReportVersion=0&pla
n.parameters.uaFalsePositiveVersion=0&plan.parameters.uaVariantDescriptionV
ersion=0&plan.parameters.uaPathogenicityVersion=0&plan.parameters.interpret
ationEtag=10075&plan.parameters.uaMrdMarkerVersion=0' \
-X 'POST' \
-H 'Content-Type: application/json' \
-H 'Accept: */*' \
-H 'Authorization: Bearer <token>' \
--data-raw '{"columns":
["\\"dynamic.sgId\\"","\\"commonVariant.commonSgId\\"","concat(\\"dynamic.s
gId\\", \\"-\\", \\"transcriptome.gene.code\\") as
unique_id","\\"transcriptome.gene.code\\"","\\"transcriptome.gene.symbol\\"
","\\"short.annotated.transcriptSelect.mappedGenesInPanel\\"","\\"transcrip
tome.transcript.name\\"","\\"transcriptome.transcript.version\\"","\\"trans
criptome.transcript.database\\"","\\"short.annotated.transcriptSelect.byFVT
\\"","\\"transcriptome.transcriptInPanel.labels.MANE\\"","\\"transcriptome.
transcriptInPanel.labels.MANE_PLUS\\"","\\"transcriptome.transcript.cdsLong
est\\"","\\"short.called.variant.location.chromosome.fullSeqName\\"","\\"sh
ort_predictor.inhouse.predictors.ABCD.result.label\\"","\\"short.annotated.
transcriptContext.transcriptVariant.cnomen.base\\"","\\"short.annotated.tra
nsriptContext.proteinVariant.pnomen.hgvsRefSeq\\"","\\"short.called.allele
Frequency\\"","\\"short.called.readDepth\\"","\\"short.annotated.transcript
Context.transcriptVariant.InExon\\"","\\"short.annotated.transcriptContext.
```

```
transcriptVariant.positionInExon\\","\\\\"short.called.variant.type\\","\\\\"
short.annotated.transcriptContext.proteinVariant.consequence.sophiaNameSele
ct\\","\\\\"short.annotated.transcriptContext.proteinVariant.consequence.sop
hiaName\\","\\\\"short.annotated.transcriptContext.proteinVariant.consequenc
e.trueIntergenic\\","\\\\"short.annotated.catalogs.gnomadGenomes.global.globa
l.value\\","\\\\"short.annotated.catalogs.gnomadExomes.global.global.value\\
\\","\\\\"userAnnotations.interpretation.acmg.result.classificationFinal\\","\\\\"
\\\\"userAnnotations.interpretation.acmg.result.score\\","\\\\"userAnnotation
s.interpretation.acmg.result.scoreFinal\\","\\\\"short.annotated.catalogs.cl
invar.CLNREVSTAT\\","\\\\"short.annotated.catalogs.clinvar.CLNSIG\\","\\\\"us
erAnnotations.account.pathogenicity.level\\","\\\\"userAnnotations.account.p
athogenicity.level\\","\\\\"userAnnotations.analysis.falsePositive.id\\","\\\\"
\\\\"userAnnotations.account.falsePositive.flagged\\","\\\\"userAnnotations.ana
lysis.falsePositive.etag\\","\\\\"userAnnotations.interpretation.inReport.fl
agged\\","\\\\"userAnnotations.analysis.falsePositive.flagged\\","\\\\"userAn
notations.account.inReport.flagged\\","\\\\"userAnnotations.account.variantD
escription.flagged\\"]],"filters":{"filterString":"fql:
(((transcriptome.gene.code anyOf
(37370,23527,8298,5585,7006,32283,24498,28413,1183,23129,31155,15907,8208,3
8546,5579,37414,6701,28094,1779,21512,39462,9156,19155,13892,36793,30171,31
024,29230,2235,30099,35387,31869,13735,34856,38283,16374,18478,15085,9244,2
0991,38520,24750,19482,30442,16939,26450,24194,10758,29842,35212,34563,3258
2,1310,656,37237,6646,4306,32546,17273,4760,40111,38872,20483,4123,18244,93
36,3750,20769,33181,40998,27835,7151,14926,2338,38918,621,105,7701,4114,372
26,39800,37942,36812,18889) AND transcriptome.transcriptInPanel.geneInPanel
= true) OR
(short.annotated.transcriptContext.proteinVariant.consequence.sophiaName =
\\'intergenic\\' AND transcriptome.gene.symbol is empty)) AND
short.called.filter.bool.PASS is true)"},"groupBy":["*"],"pagination":
{"offset":50,"limit":50},"columnSorts":
[{"column":"\\\\"short_predictor.inhouse.predictors.ABCD.result.label\\","so
rt":"ASC"}, {"column":"\\\\"dynamic.sgId\\","sort":"ASC"}]}'
```

Based on this, you can test the query in the terminal, and modify the body as defined below.

Base URL

All endpoints below are served under the configurable context path:

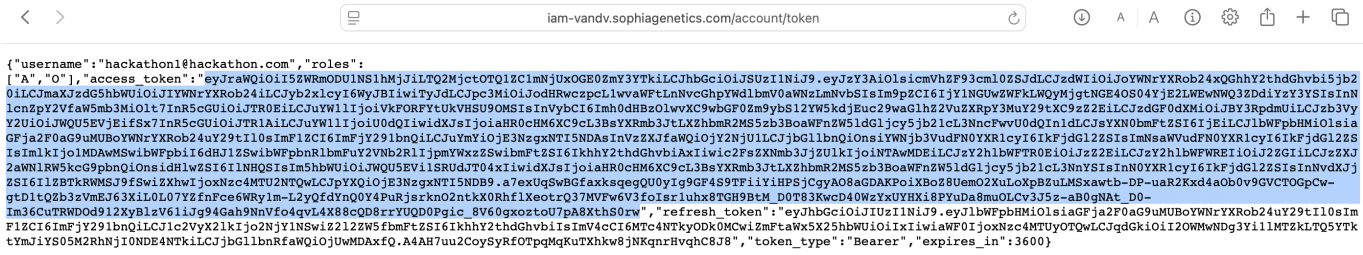
```
http://<host>/api/variant/query
```

Authentication

Every endpoint requires authentication. Pass a bearer token in the **Authorization** header:

```
Authorization: Bearer <IAM_TOKEN>
```

The IAM_TOKEN you can get from <https://iam-vandv.sophiagenetics.com/account/token> , field `access_token`, or by checking the requests sent from the front-end to the service. Any token there will be valid for 1 hour.



Common request parameters (you shouldn't have to change them)

All endpoints accept the same `parameters` query object (flattened as `parameters.*` query string entries):

Field	Type	Required	Description
<code>parameters.key</code>	string	yes	The dataset key (resolved by the configured <code>DatasetProvider</code>).
<code>parameters.plan.transformers</code>	string	no	Comma-separated list of <code>DataTransformerKey</code> values applied to the base dataset, in order. Modifies the data by joining external data sources.
<code>parameters.plan.parameters.*</code>	map	no	Free-form parameters consumed by the transformers (e.g. <code>analysisId</code> , <code>interpretationId</code> , version flags...).
<code>parameters.engine.*</code>	object	no	Optional engine tuning parameters.

The `Query` body

The POST endpoints take a JSON `Query` body:

```
{
  "columns":    ["*"],
  "groupBy":    null,
  "filters":    { "filterString": "" },
  "columnSorts": [
    { "column": "chr", "sort": "ASC", "nullsFirst": true }
  ],
  "pagination": { "offset": 0, "limit": 100 }
}
```

This translates in the backend to an SQL that is executed on the data. Please make sure to escape any columns that contain a dot or another special character, for example
`"\"short.annotated.transcriptContext.proteinVariant.consequence.trueIntergenic\""`.

Field	Type	Notes
<code>columns</code>	<code>string[]</code>	Columns to return. Use <code>["*"]</code> for all. The controller currently always uses default columns server-side.
<code>groupBy</code>	<code>string[]</code>	Optional grouping columns.
<code>filters.filterString</code>	<code>string</code>	Free-form filter expression in a custom FQL format (not SQL, to prevent SQL injections).
<code>columnSorts[]</code>	<code>array</code>	<code>column + sort (ASC/DESC) + optional nullsFirst</code> .
<code>pagination.offset</code>	<code>int</code>	Defaults to <code>0</code> .
<code>pagination.limit</code>	<code>int</code>	Defaults to <code>1000</code> if omitted.

Filter expression syntax (FQL)

`filters.filterString` accepts an FQL expression, prefixed with `fql:`. Each leaf is `column <operator> <value>`; leaves are combined with `AND` / `OR` and grouped with parentheses. String values use single quotes; column names with dots must be quoted (e.g. `"short.called.alleleFrequency"`). String comparisons are case-insensitive.

Append `?` to any operator (e.g. `=?`, `anyOf?`, `between?`) to also match `NULL` rows.

Operator	Meaning	Example
<code>=, !=</code>	equals / not equals	<code>gene.code = 'BRCA1'</code>
<code><, <=, >, >=</code>	numeric comparison	<code>alleleFrequency >= 0.05</code>
<code>~, !~</code>	contains / does not contain (substring)	<code>"gene.symbol" ~ 'BRCA'</code>
<code>=~, ~=</code>	starts with / ends with	<code>transcript.name ~= 'NM_'</code>
<code>is empty, is not empty</code>	NULL check	<code>pathogenicity is not empty</code>
<code>anyOf (...), noneOf (...)</code>	value in / not in list	<code>"gene.code" anyOf (123, 456)</code>
<code>between (min, max), outside (min, max)</code>	numeric range	<code>readDepth between (20, 200)</code>
<code>anyOfList, containsAllOf, allOf (and not... variants)</code>	match against composite/array string columns	<code>labels anyOfList ('mane', 'clinvar')</code>

Combine with **AND** / **OR** and parentheses:

```
fql:(("gene.code anyOf" (123,456) AND "transcriptInPanel.geneInPanel" =
true)
  OR "consequence.sophiaName" = 'intergenic')
AND "filter.bool.PASS" is true
```

Endpoints

1. **GET** /schema

Return the column schema of the resolved dataset (after applying transformers).

Query params: the standard **parameters.*** block above.

Response 200 — DataSchema:

```
{ "columns": ["chr", "pos", "ref", "alt", "..."] }
```

Example:

```
curl -G "https://platform-
vandv1.sophiagenetics.com/api/variant/query/schema" \
--data-urlencode "parameters.key=user_annotations" \
--data-urlencode
"parameters.plan.transformers=USER_ANNOTATIONS,MATERIALIZE" \
-H "Authorization: Bearer $IAM_TOKEN"
```

2. **POST** / — paginated query

Run a single query against the dataset and return one page of results.

Query params: **parameters.*** **Body:** a **Query** (see above).

Response 200 — Page:

```
{
  "header": {
    "offset": 0,
    "key": "user_annotations",
  },
  "pageContent": {
    "columns": ["chr", "pos", "ref", "alt"],
    "data":    [["1", 12345, "A", "G"], ["1", 23456, "C", "T"]]
```

```
}  
}
```

Example:

```
curl -X POST "https://platform-vandv1.sophiagenetics.com/api/variant/query?  
\  
parameters.key=user_annotations&\  
parameters.plan.transformers=USER_ANNOTATIONS&\  
parameters.plan.parameters.analysisId=1&\  
parameters.plan.parameters.interpretationId=1&\  
parameters.plan.parameters.experimentTypeId=126000" \  
-H "Authorization: Bearer $IAM_TOKEN" \  
-H "Content-Type: application/json" \  
--data '{  
  "columns": ["*"],  
  "filters": { "filterString": "" },  
  "columnSorts": [],  
  "pagination": { "offset": 0, "limit": 100 }  
}'
```